Data Enc: Data Preprocessing – Skewness, Scaling and Encoding

Aim

Study skewness, apply Min-Max scaling and standardization, and encode ordinal and nominal categorical features using the urban-climate dataset.

Dataset

`urban_climate.csv` contains 11,040 records and 20 attributes. The dataset includes numerical and categorical features.

1. Load and inspect the dataset

Load the CSV file and inspect its first rows, shape, data types and missing values.

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import MinMaxScaler, StandardScaler

df = pd.read_csv('urban_climate.csv')

print(df.head())
print('Shape:', df.shape)
print(df.dtypes)
print(df.isnull().sum())
```

```
       city country  latitude  longitude  year  month  temperature_celsius  \
0  New York     USA   40.7128    -74.006  1980      1                 6.21
1  New York     USA   40.7128    -74.006  1980      2                17.46
2  New York     USA   40.7128    -74.006  1980      3                20.28
3  New York     USA   40.7128    -74.006  1980      4                22.31
4  New York     USA   40.7128    -74.006  1980      5                20.01

   humidity_percent  precipitation_mm  wind_speed_ms  \
0              48.8              29.5            8.6
1              38.7              23.2            7.6
2              32.3              30.7            4.6
3              30.6              26.9           10.1
4              22.6              57.8           10.7

   urban_heat_island_intensity           data_source  \
0                          2.0  NOAA_Weather_Stations
1                          2.0  NOAA_Weather_Stations
2                          2.0  NOAA_Weather_Stations
3                          2.0  NOAA_Weather_Stations
4                          2.0  NOAA_Weather_Stations

   country_population_reference  country_area_km2 continent         subregion  \
0                     319259000           9629091  Americas  Northern America
1                     319259000           9629091  Americas  Northern America
2                     319259000           9629091  Americas  Northern America
3                     319259000           9629091  Americas  Northern America
4                     319259000           9629091  Americas  Northern America

      capital_city  calling_code iso3_code currency
0  Washington D.C.             1       USA      USD
1  Washington D.C.             1       USA      USD
2  Washington D.C.             1       USA      USD
3  Washington D.C.             1       USA      USD
4  Washington D.C.             1       USA      USD
Shape: (11040, 20)
city                              str
country                           str
latitude                      float64
longitude                     float64
year                            int64
month                           int64
temperature_celsius           float64
humidity_percent              float64
precipitation_mm              float64
wind_speed_ms                 float64
urban_heat_island_intensity   float64
data_source                       str
country_population_reference    int64
country_area_km2                int64
continent                         str
subregion                         str
capital_city                      str
calling_code                    int64
iso3_code                         str
currency                          str
dtype: object
city                          0
country                       0
latitude                      0
longitude                     0
```

```
year                           0
month                          0
temperature_celsius            0
humidity_percent               0
precipitation_mm               0
wind_speed_ms                  0
urban_heat_island_intensity    0
data_source                    0
country_population_reference   0
country_area_km2               0
continent                      0
subregion                      0
capital_city                   0
calling_code                   0
iso3_code                      0
currency                       0
dtype: int64
```

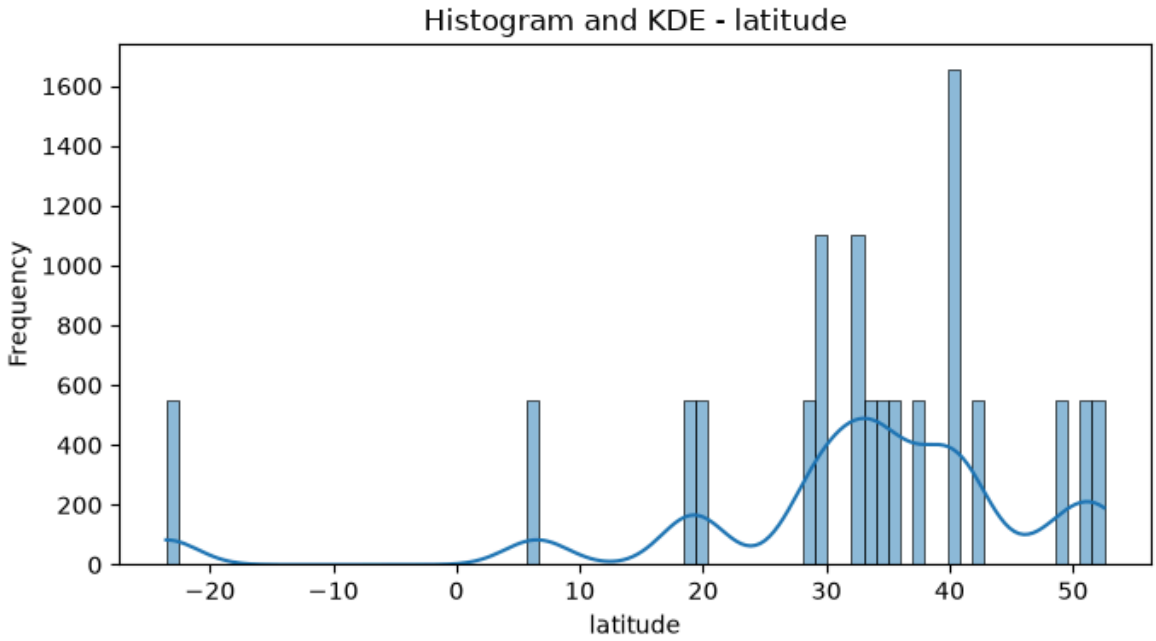
2. Skewness, histogram and KDE

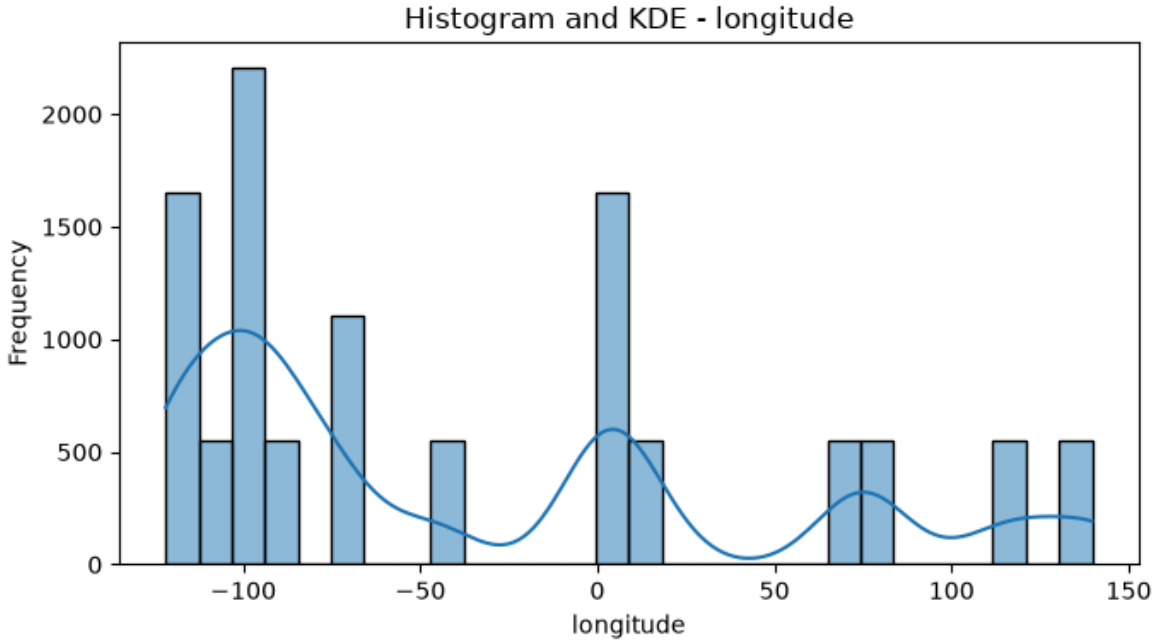
Skewness indicates asymmetry in a numerical distribution. A positive value indicates a longer right tail; a negative value indicates a longer left tail. Histograms and KDE curves help visualize the distributions.

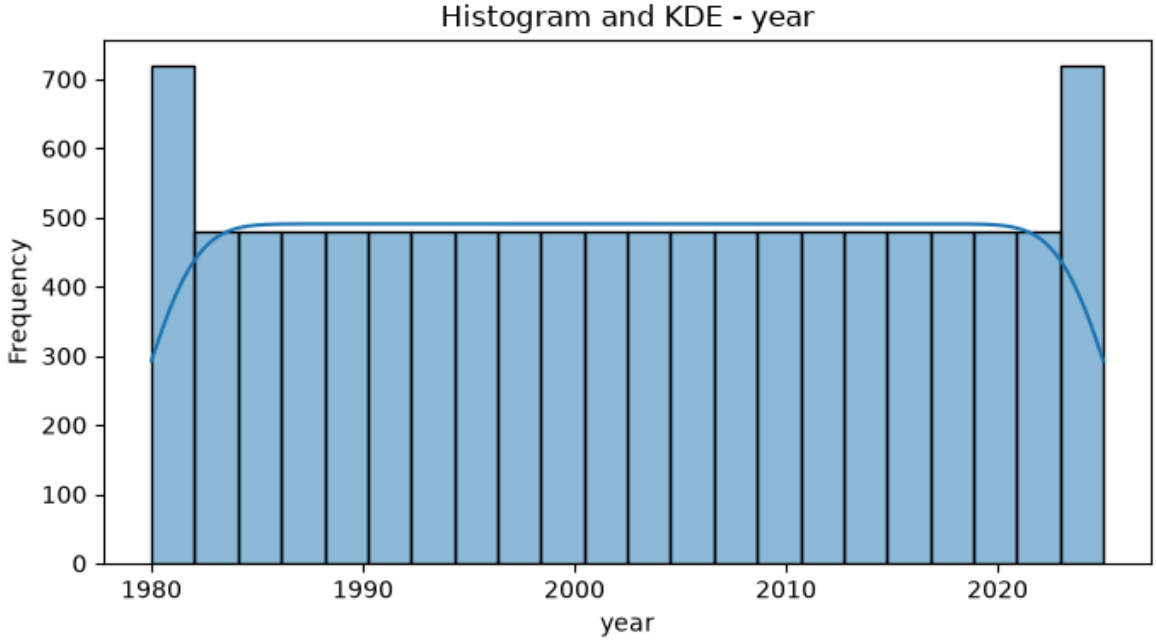
In [2]:
```
numeric_cols = df.select_dtypes(include=np.number).columns.tolist()
skewness = df[numeric_cols].skew()
print(skewness)

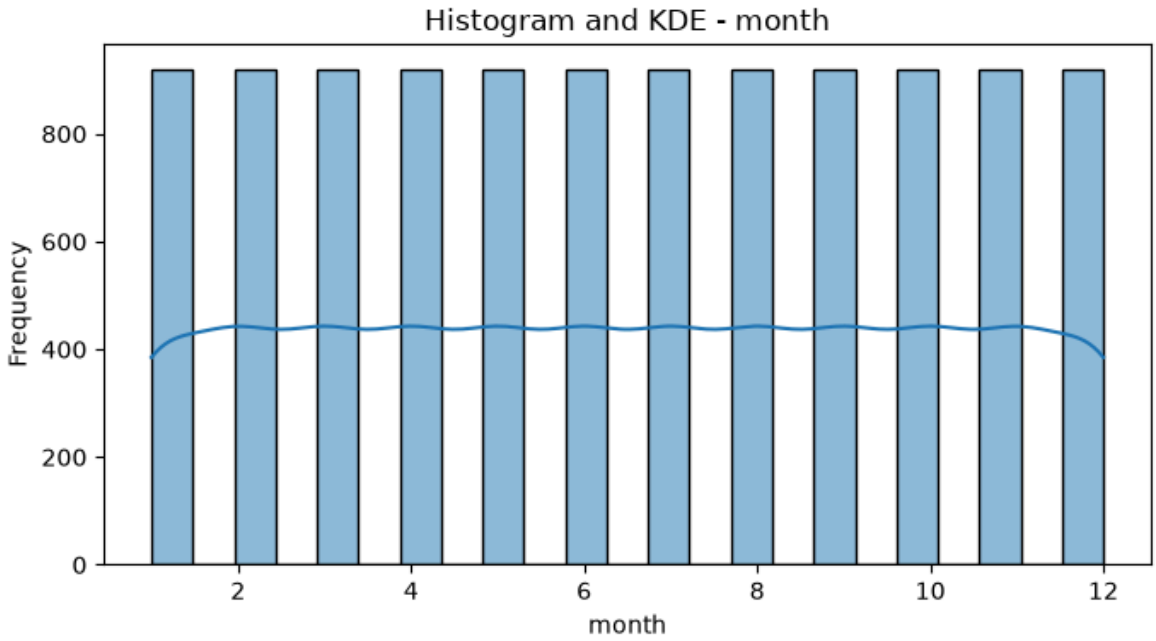
for col in numeric_cols:
    plt.figure(figsize=(7, 4))
    sns.histplot(df[col].dropna(), kde=True)
    plt.title('Histogram and KDE - ' + col)
    plt.xlabel(col)
    plt.ylabel('Frequency')
    plt.tight_layout()
    plt.show()
```

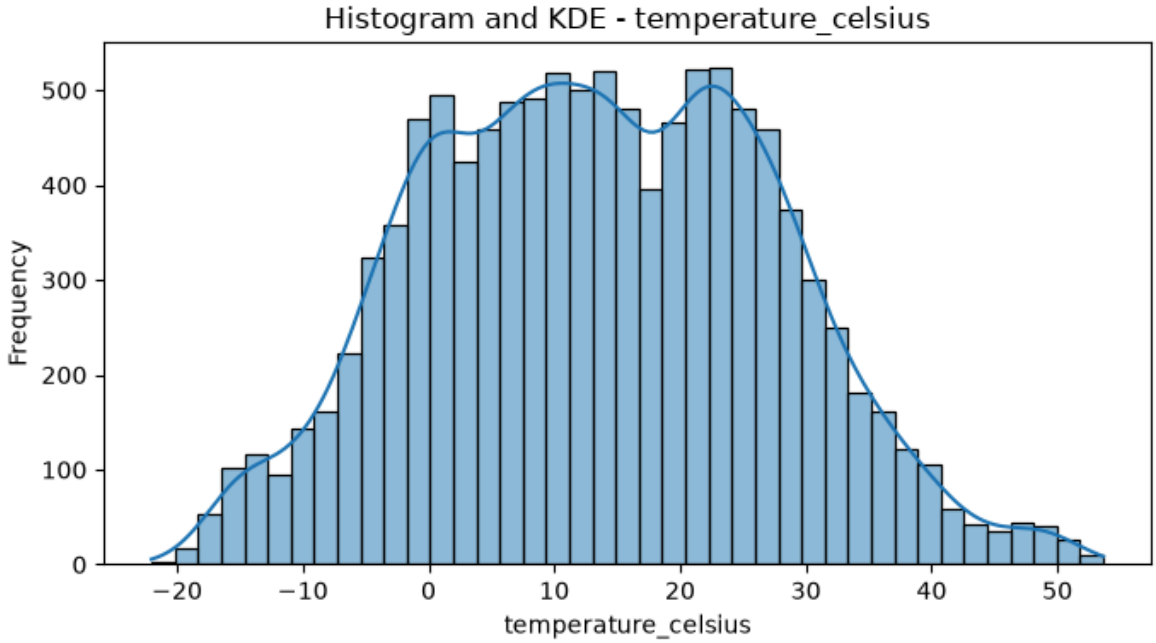
```
latitude                      -1.757180
longitude                      0.792818
year                           0.000000
month                          0.000000
temperature_celsius            0.084897
humidity_percent               0.073614
precipitation_mm               0.789653
wind_speed_ms                  0.020291
urban_heat_island_intensity    1.154857
country_population_reference   1.716200
country_area_km2              -0.497278
calling_code                   1.984061
dtype: float64
```

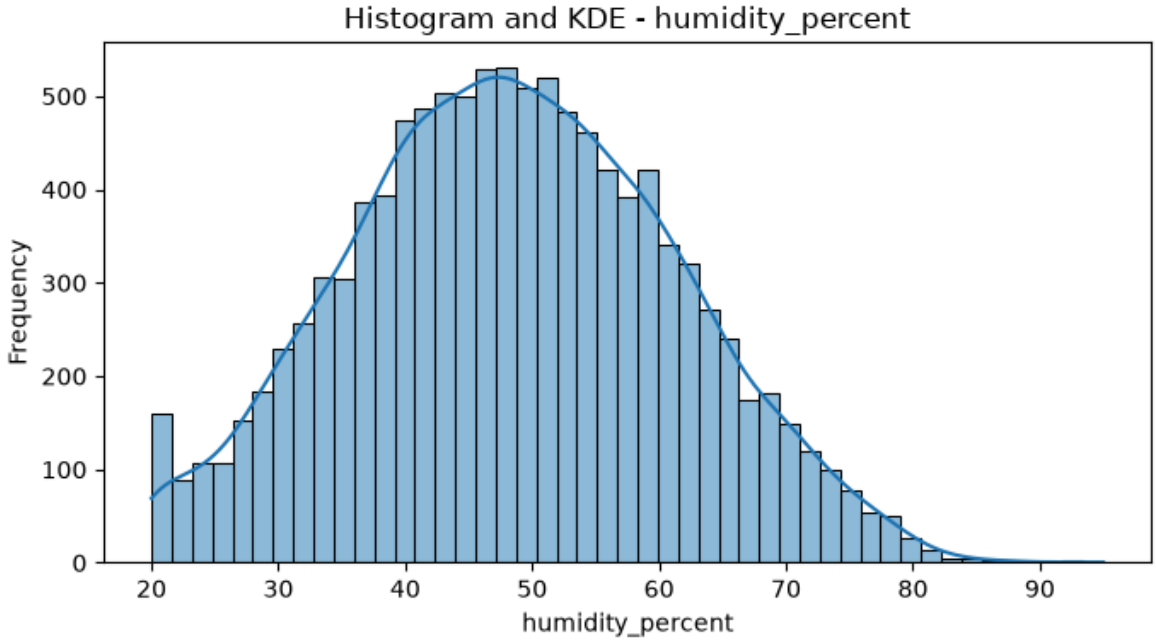

Histogram and KDE - latitude
Frequency
latitude
0
200
400
600
800
1000
1200
1400
1600
−20
−10
0
10
20
30
40
50

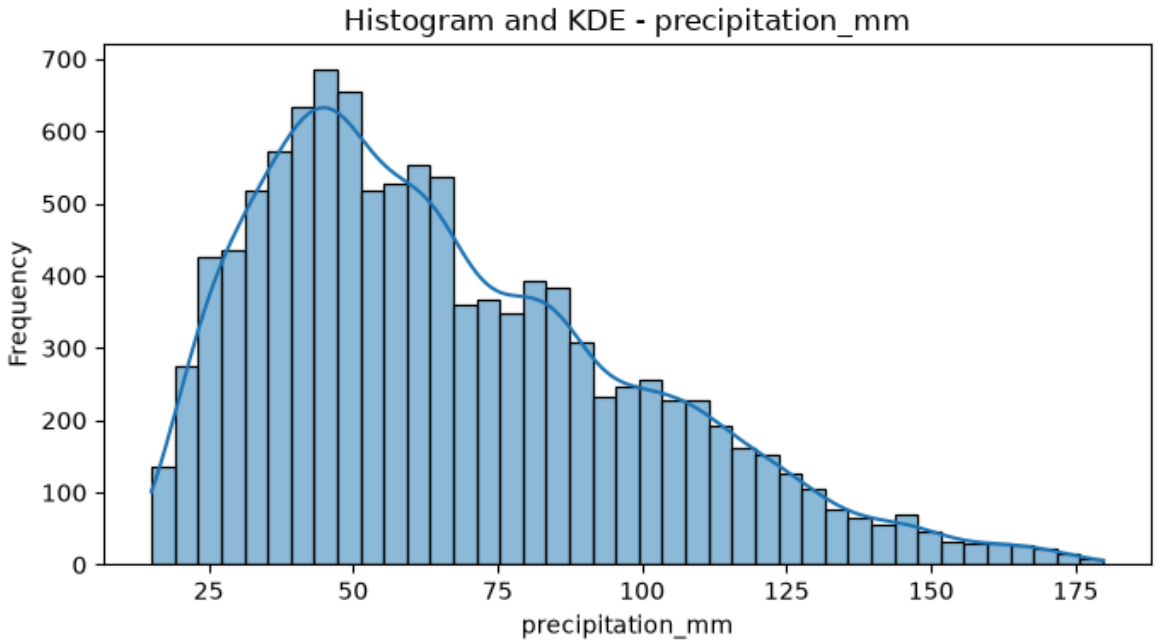

Histogram and KDE - longitude
Frequency
longitude
0
500
1000
1500
2000
−100
−50
0
50
100
150

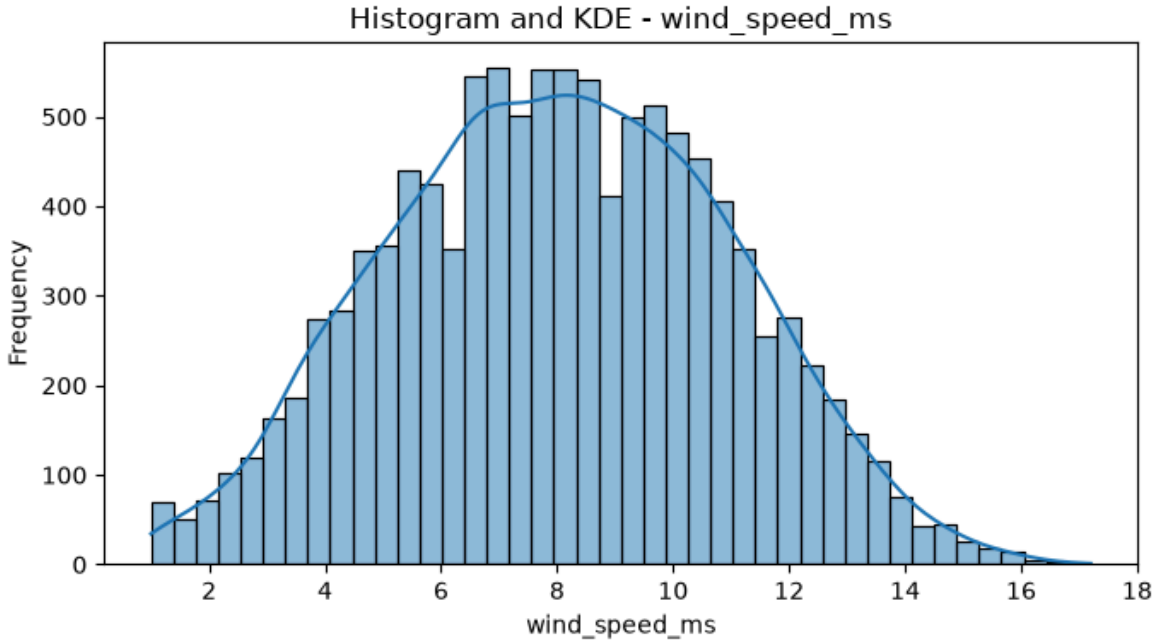

Histogram and KDE - year
Frequency
year
0
100
200
300
400
500
600
700
1980
1990
2000
2010
2020

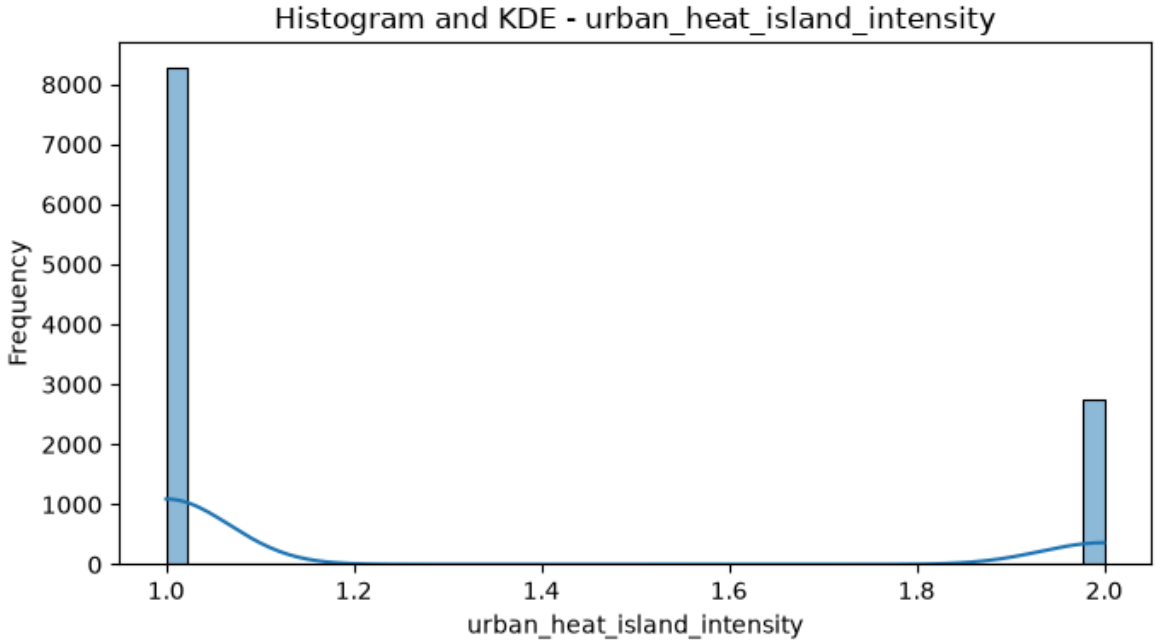

Histogram and KDE - month
Frequency
0
200
400
600
800
2
4
6
8
10
12
month

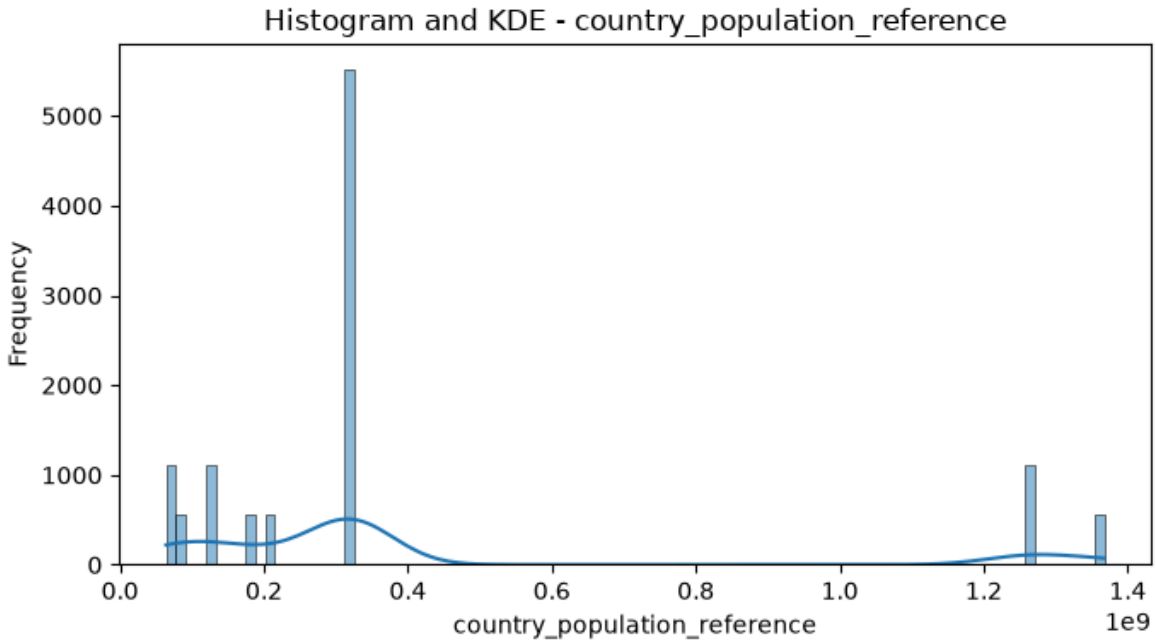

Histogram and KDE - temperature_celsius
Frequency
0
100
200
300
400
500
−20
−10
0
10
20
30
40
50
temperature_celsius

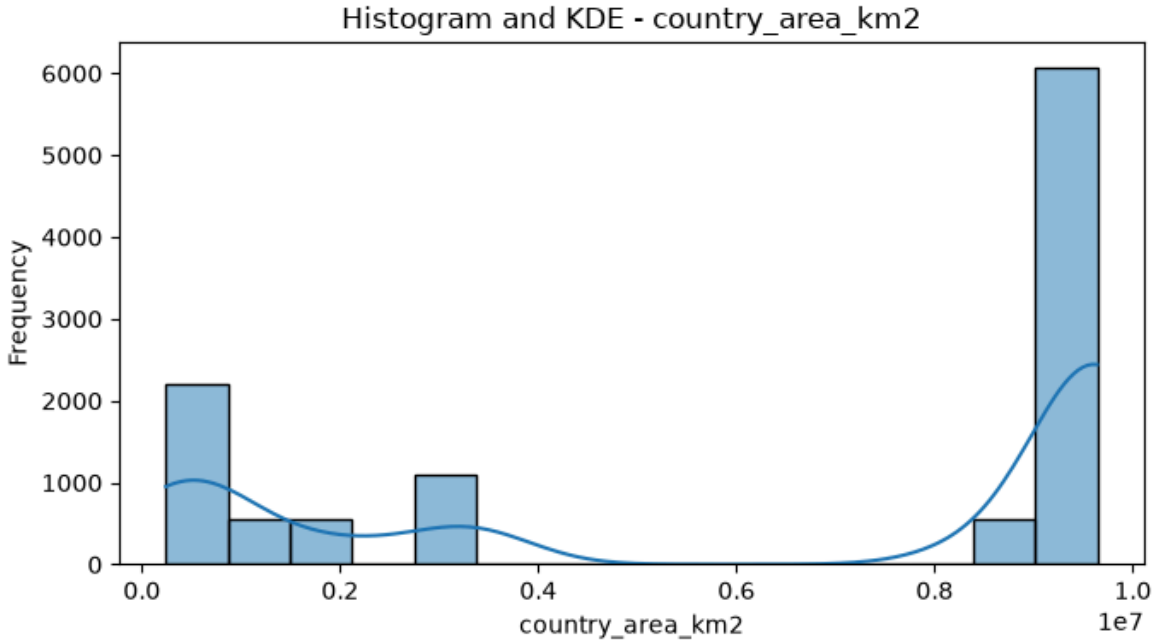

Histogram and KDE - humidity_percent
Frequency
0
100
200
300
400
500
20
30
40
50
60
70
80
90
humidity_percent

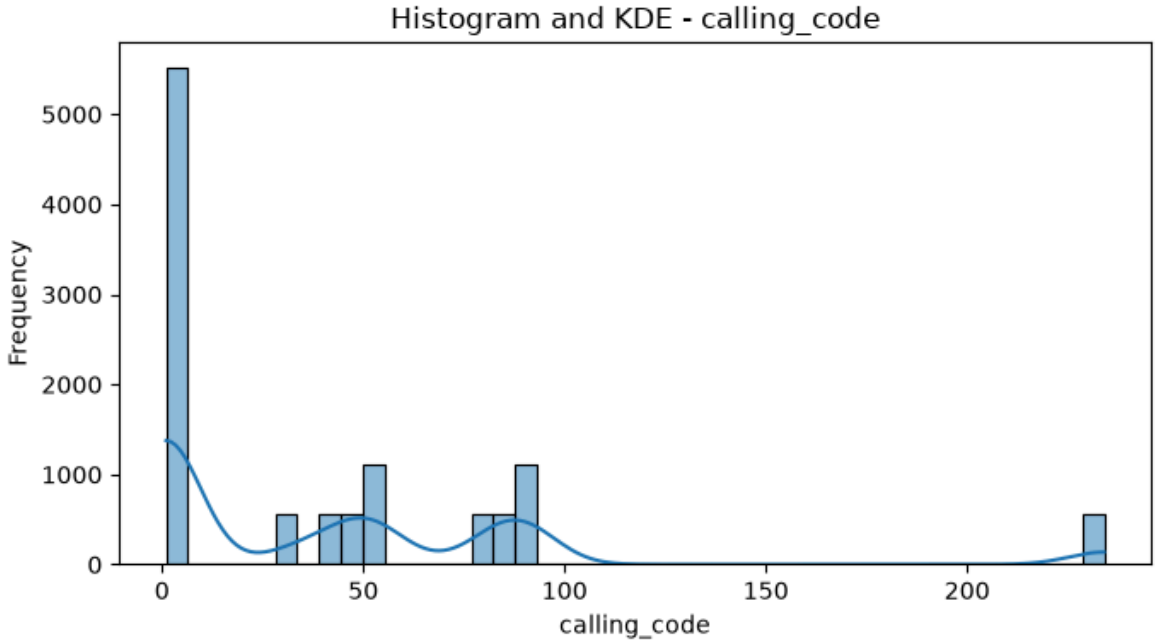

Histogram and KDE - precipitation_mm
Frequency
precipitation_mm
0
100
200
300
400
500
600
700
25
50
75
100
125
150
175


Histogram and KDE - wind_speed_ms
Frequency
wind_speed_ms
0
100
200
300
400
500
2
4
6
8
10
12
14
16
18


Histogram and KDE - urban_heat_island_intensity
Frequency
urban_heat_island_intensity
0
1000
2000
3000
4000
5000
6000
7000
8000
1.0
1.2
1.4
1.6
1.8
2.0


Histogram and KDE - country_population_reference
Frequency
0
1000
2000
3000
4000
5000
0.0
0.2
0.4
0.6
0.8
1.0
1.2
1.4
country_population_reference
1e9


Histogram and KDE - country_area_km2
Frequency
0
1000
2000
3000
4000
5000
6000
0.0
0.2
0.4
0.6
0.8
1.0
country_area_km2
1e7


Histogram and KDE - calling_code
Frequency
0
1000
2000
3000
4000
5000
0
50
100
150
200
calling_code

3. Min-Max scaling manually and with Scikit-Learn

Formula: `X_scaled = (X - X_min) / (X_max - X_min)`. The manual Pandas implementation is compared with `MinMaxScaler`.

In [3]:
```
scale_cols = ['temperature_celsius', 'humidity_percent', 'precipitation_mm', 'wi
scale_cols = [c for c in scale_cols if c in df.columns]

manual_scaled = df[scale_cols].copy()
for col in scale_cols:
    minimum, maximum = df[col].min(), df[col].max()
    manual_scaled[col] = (df[col] - minimum) / (maximum - minimum) if maximum !=

minmax_scaler = MinMaxScaler()
sklearn_scaled = pd.DataFrame(minmax_scaler.fit_transform(df[scale_cols]), colum
print('Manual scaling:')
print(manual_scaled.head())
print('Scikit-Learn scaling:')
print(sklearn_scaled.head())
print('Minimum values:')
print(sklearn_scaled.min())
print('Maximum values:')
print(sklearn_scaled.max())
print('Maximum difference between manual and sklearn:', (manual_scaled - sklearn
```

```
Manual scaling:
   temperature_celsius  humidity_percent  precipitation_mm  wind_speed_ms  \
0             0.373150          0.384000          0.087432       0.469136
1             0.521802          0.249333          0.049180       0.407407
2             0.559064          0.164000          0.094718       0.222222
3             0.585888          0.141333          0.071645       0.561728
4             0.555497          0.034667          0.259259       0.598765

   urban_heat_island_intensity
0                          1.0
1                          1.0
2                          1.0
3                          1.0
4                          1.0
Scikit-Learn scaling:
   temperature_celsius  humidity_percent  precipitation_mm  wind_speed_ms  \
0             0.373150          0.384000          0.087432       0.469136
1             0.521802          0.249333          0.049180       0.407407
2             0.559064          0.164000          0.094718       0.222222
3             0.585888          0.141333          0.071645       0.561728
4             0.555497          0.034667          0.259259       0.598765

   urban_heat_island_intensity
0                          1.0
1                          1.0
2                          1.0
3                          1.0
4                          1.0
Minimum values:
temperature_celsius            0.0
humidity_percent               0.0
precipitation_mm               0.0
wind_speed_ms                  0.0
urban_heat_island_intensity    0.0
dtype: float64
Maximum values:
temperature_celsius            1.0
humidity_percent               1.0
precipitation_mm               1.0
wind_speed_ms                  1.0
urban_heat_island_intensity    1.0
dtype: float64
Maximum difference between manual and sklearn: 2.220446049250313e-16
```

4. Ordinal encoding

The source dataset does not contain a pre-existing ordinal feature such as education level. A derived precipitation category is created from precipitation quantiles solely to demonstrate ordinal encoding: Low = 0, Moderate = 1, High = 2.

In [4]:
```
df_encoded = df.copy()
df_encoded['precipitation_level'] = pd.qcut(
    df_encoded['precipitation_mm'].rank(method='first'),
    q=3,
    labels=['Low', 'Moderate', 'High']
)
ordinal_mapping = {'Low': 0, 'Moderate': 1, 'High': 2}
```

```
df_encoded['precipitation_level_encoded'] = df_encoded['precipitation_level'].ma
print(df_encoded[['precipitation_mm', 'precipitation_level', 'precipitation_leve
```

```
   precipitation_mm precipitation_level  precipitation_level_encoded
0              29.5                 Low                            0
1              23.2                 Low                            0
2              30.7                 Low                            0
3              26.9                 Low                            0
4              57.8            Moderate                            1
```

5. One-Hot encoding

Nominal categories have no natural order. `get_dummies()` converts them into indicator columns. `drop_first=True` removes one reference category per feature to avoid redundant dummy columns (the Dummy Variable Trap in a full-rank linear model with an intercept).

In [5]:
```
nominal_cols = [c for c in ['city', 'country', 'continent', 'subregion', 'data_s
encoded_df = pd.get_dummies(df_encoded, columns=nominal_cols, drop_first=True, d
print('Nominal columns encoded:', nominal_cols)
print(encoded_df.head())
```

```
Nominal columns encoded: ['city', 'country', 'continent', 'subregion', 'data_sour
ce', 'capital_city', 'iso3_code', 'currency']
   latitude  longitude  year  month  temperature_celsius  humidity_percent  \
0   40.7128    -74.006  1980      1                 6.21              48.8
1   40.7128    -74.006  1980      2                17.46              38.7
2   40.7128    -74.006  1980      3                20.28              32.3
3   40.7128    -74.006  1980      4                22.31              30.6
4   40.7128    -74.006  1980      5                20.01              22.6

   precipitation_mm  wind_speed_ms  urban_heat_island_intensity  \
0              29.5            8.6                          2.0
1              23.2            7.6                          2.0
2              30.7            4.6                          2.0
3              26.9           10.1                          2.0
4              57.8           10.7                          2.0

   country_population_reference  ...  iso3_code_NGA  iso3_code_USA  \
0                     319259000  ...              0              1
1                     319259000  ...              0              1
2                     319259000  ...              0              1
3                     319259000  ...              0              1
4                     319259000  ...              0              1

  currency_CNY  currency_EUR  currency_GBP  currency_INR  currency_JPY  \
0            0             0             0             0             0
1            0             0             0             0             0
2            0             0             0             0             0
3            0             0             0             0             0
4            0             0             0             0             0

   currency_MXN  currency_NGN  currency_USD
0             0             0             1
1             0             0             1
2             0             0             1
3             0             0             1
4             0             0             1

[5 rows x 78 columns]
```

6. Standardization and comparison

Standardization uses `z = (x - mean) / population_std`. `StandardScaler` produces approximately mean 0 and population standard deviation 1. The first five rows and shapes of the original and transformed datasets are displayed.

In [6]:
```
standardized_df = encoded_df.copy()
standard_scaler = StandardScaler()
standardized_df[numeric_cols] = standard_scaler.fit_transform(standardized_df[nu

print('Standardized means:')
print(standardized_df[numeric_cols].mean().round(4))
print('Standardized population standard deviations:')
print(standardized_df[numeric_cols].std(ddof=0).round(4))

print('Original dataset:')
print(df.head())
print('Transformed dataset:')
print(standardized_df.head())
```

```
print('Original shape:', df.shape)
print('Transformed shape:', standardized_df.shape)

normalized_df = df.copy()
normalized_df[scale_cols] = sklearn_scaled
normalized_df.to_csv('urban_climate_minmax_scaled.csv', index=False)
standardized_df.to_csv('urban_climate_preprocessed.csv', index=False)
print('Saved transformed CSV files.')
```

```
Standardized means:
latitude                       -0.0
longitude                       0.0
year                           -0.0
month                           0.0
temperature_celsius             0.0
humidity_percent                0.0
precipitation_mm                0.0
wind_speed_ms                   0.0
urban_heat_island_intensity    -0.0
country_population_reference    0.0
country_area_km2               -0.0
calling_code                    0.0
dtype: float64
Standardized population standard deviations:
latitude                        1.0
longitude                       1.0
year                            1.0
month                           1.0
temperature_celsius             1.0
humidity_percent                1.0
precipitation_mm                1.0
wind_speed_ms                   1.0
urban_heat_island_intensity     1.0
country_population_reference    1.0
country_area_km2                1.0
calling_code                    1.0
dtype: float64
Original dataset:
       city country  latitude  longitude  year  month  temperature_celsius  \
0  New York     USA   40.7128    -74.006  1980      1                 6.21
1  New York     USA   40.7128    -74.006  1980      2                17.46
2  New York     USA   40.7128    -74.006  1980      3                20.28
3  New York     USA   40.7128    -74.006  1980      4                22.31
4  New York     USA   40.7128    -74.006  1980      5                20.01

   humidity_percent  precipitation_mm  wind_speed_ms  \
0              48.8              29.5            8.6
1              38.7              23.2            7.6
2              32.3              30.7            4.6
3              30.6              26.9           10.1
4              22.6              57.8           10.7

   urban_heat_island_intensity           data_source  \
0                          2.0  NOAA_Weather_Stations
1                          2.0  NOAA_Weather_Stations
2                          2.0  NOAA_Weather_Stations
3                          2.0  NOAA_Weather_Stations
4                          2.0  NOAA_Weather_Stations

   country_population_reference  country_area_km2 continent         subregion  \
0                     319259000           9629091  Americas  Northern America
1                     319259000           9629091  Americas  Northern America
2                     319259000           9629091  Americas  Northern America
3                     319259000           9629091  Americas  Northern America
4                     319259000           9629091  Americas  Northern America

      capital_city  calling_code iso3_code currency
0  Washington D.C.             1       USA      USD
1  Washington D.C.             1       USA      USD
```

```
2  Washington D.C.             1       USA      USD
3  Washington D.C.             1       USA      USD
4  Washington D.C.             1       USA      USD
Transformed dataset:
   latitude  longitude      year     month  temperature_celsius  \
0  0.552787  -0.467809 -1.694798 -1.593255            -0.525341
1  0.552787  -0.467809 -1.694798 -1.303572             0.283025
2  0.552787  -0.467809 -1.694798 -1.013890             0.485656
3  0.552787  -0.467809 -1.694798 -0.724207             0.631521
4  0.552787  -0.467809 -1.694798 -0.434524             0.466255

   humidity_percent  precipitation_mm  wind_speed_ms  \
0          0.028695         -1.145124       0.193535
1         -0.760872         -1.338475      -0.149362
2         -1.261191         -1.108295      -1.178052
3         -1.394088         -1.224920       0.707880
4         -2.019487         -0.276578       0.913618

   urban_heat_island_intensity  country_population_reference  ...  \
0                     1.732051                     -0.197145  ...
1                     1.732051                     -0.197145  ...
2                     1.732051                     -0.197145  ...
3                     1.732051                     -0.197145  ...
4                     1.732051                     -0.197145  ...

   iso3_code_NGA  iso3_code_USA currency_CNY  currency_EUR  currency_GBP  \
0              0              1            0             0             0
1              0              1            0             0             0
2              0              1            0             0             0
3              0              1            0             0             0
4              0              1            0             0             0

   currency_INR  currency_JPY  currency_MXN  currency_NGN  currency_USD
0             0             0             0             0             1
1             0             0             0             0             1
2             0             0             0             0             1
3             0             0             0             0             1
4             0             0             0             0             1

[5 rows x 78 columns]
Original shape: (11040, 20)
Transformed shape: (11040, 78)
Saved transformed CSV files.
```

Conclusion

The experiment inspected numerical distributions using skewness, histograms and KDE. Min-Max scaling was implemented manually and with Scikit-Learn. A derived ordinal precipitation feature demonstrated ordered label mapping, while nominal features were one-hot encoded with `drop_first=True`. StandardScaler was used to verify mean approximately 0 and population standard deviation approximately 1. Min-Max scaled features were verified to have minimum 0 and maximum 1.